[public 0.6] Baseline 코드 및 insight 공유

저작운동

2025.12.29 22:21 914 조회

public score 기준 약 0.6 수준의 전처리 중심 파이프라인과 관련된 코드를 공유합니다. (모델은 baseline 모델과 동일합니다.)

성능 자체보다는 이 대회의 구조와 규칙을 이해했을 때 어디서부터 시작하는 것이 자연스러운지에 대한 큰 그림을 설계하는 참고용으로 봐주시면 좋겠습니다.

이번 대회는 공식 학습 데이터가 없고 프레임 단위 입력과 단일 모델 사용 등 제약이 명확한 만큼 모델 이전 단계에서 입력을 어떻게 정리할지 고민하는 과정이 중요하다

고 느꼈습니다.
특히 딥페이크 탐지에서는 이미지 전체보다 얼굴 중심으로 입력을 정리하는 것만으로도 파이프라인이 훨씬 안정됩니다.

이미 forgery detection 분야에 능통하신 분들이 많다고 생각해 공유가 조심스럽지만,
이 글과 코드는 어디서부터 시작해야 할지 막막한 분들이 전체 흐름을 빠르게 파악하는 데 도움이 되었으면 하는 마음으로 올립니다.

코드

다운로드

Pre-processing + Baseline ViT Pipeline

pre-processing flow

1. **face detecting** - Using dlib
2. **landmark detecting** - 81개 랜드마크 중 5개 core points 추출
3. **face alignment** - SimilarityTransform으로 정렬 후 224x224로 crop
4. **face crop saving** - 500개 샘플 전부 저장(crop checking)
5. **ViT model inference** - baseling model

env

python == 3.9
dlib (conda install dlib)
torch == 2.8.0+cu128
이외 설치는 pip install 활용해 설치 (라이브러리 설치 error는 댓글로 문의.)

Import

```
import os
import random
import numpy as np
import pandas as pd
from pathlib import Path
from typing import Dict, List, Optional, Tuple

import cv2
import dlib
import torch
import torch.nn.functional as F
```

```
from PIL import Image
from tqdm import tqdm
from transformers import ViTForImageClassification, ViTImageProcessor
from skimage import transform as trans
```

```
/home/army/miniconda/envs/detector/lib/python3.9/site-packages/tqdm/auto.py:21: TqdmWarning: IProgress not found. Please update jupyter
and ipywidgets. See https://ipywidgets.readthedocs.io/en/stable/user_install.html
  from .autonotebook import tqdm as notebook_tqdm
```

Settings

```
SEED = 42
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)
torch.cuda.manual_seed_all(SEED)

torch.backends.cudnn.deterministic = True
torch.backends.cudnn.benchmark = False
```

```
MODEL_ID = "prithivMLmods/Deep-Fake-Detector-v2-Model"
TEST_DIR = Path("./test_data")

# Landmark model path
# Download from: https://huggingface.co/spaces/liangtian/birthdayCrown/blob/main/shape_predictor_81_face_landmarks.dat
LANDMARK_MODEL_PATH = Path("./preprocessing/shape_predictor_81_face_landmarks.dat")

# Output directories
OUTPUT_DIR = Path("./output")
```

```
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

# 전처리 이미지 저장 여부
SAVE_CROPS = False  # If you wandt to save cropped face images

# Cropped faces directory
CROP_SAVE_DIR = OUTPUT_DIR / "cropped_faces"
CROP_SAVE_DIR.mkdir(parents=True, exist_ok=True)

OUT_CSV = OUTPUT_DIR / "submission.csv"
```

```
IMAGE_EXTS = {".jpg", ".jpeg", ".png", ".jfif"}
VIDEO_EXTS = {".mp4", ".mov"}

TARGET_SIZE = (224, 224)  # Face crop
NUM_FRAMES = 10  # 비디오 샘플링 프레임 수

DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
print(f"Device: {DEVICE}")
```

```
Device: cuda
```

Face Detection & Alignment

```
# Load dlib models
if not LANDMARK_MODEL_PATH.exists():
    raise FileNotFoundError(
        f"Landmark model not found: {LANDMARK_MODEL_PATH}\n"
        "Please download shape_predictor_81_face_landmarks.dat"
```

```
    )

face_detector = dlib.get_frontal_face_detector()
landmark_predictor = dlib.shape_predictor(str(LANDMARK_MODEL_PATH))
```

```
Loading dlib face detector and landmark predictor...
Face detector and landmark predictor loaded.
```

```
def get_5_keypoints(image_rgb: np.ndarray, face: dlib.rectangle) -> np.ndarray:
    """
    81개 랜드마크에서 5개의 core point 추출
    - left eye (#37), right eye (#44), nose (#30)
    - left mouth (#49), right mouth (#55)
    """
    shape = landmark_predictor(image_rgb, face)

    leye = np.array([shape.part(37).x, shape.part(37).y]).reshape(-1, 2)
    reye = np.array([shape.part(44).x, shape.part(44).y]).reshape(-1, 2)
    nose = np.array([shape.part(30).x, shape.part(30).y]).reshape(-1, 2)
    lmouth = np.array([shape.part(49).x, shape.part(49).y]).reshape(-1, 2)
    rmouth = np.array([shape.part(55).x, shape.part(55).y]).reshape(-1, 2)

    pts = np.concatenate([leye, reye, nose, lmouth, rmouth], axis=0)
    return pts


def align_and_crop_face(img_rgb: np.ndarray, landmarks: np.ndarray,
                        outsize: Tuple[int, int] = (224, 224),
                        scale: float = 1.3) -> np.ndarray:
    """
    5개 랜드마크를 사용하여 얼굴 정렬 및 crop
```

```
    """
    target_size = [112, 112]
    dst = np.array([
        [30.2946, 51.6963],
        [65.5318, 51.5014],
        [48.0252, 71.7366],
        [33.5493, 92.3655],
        [62.7299, 92.2041]
    ], dtype=np.float32)

    if target_size[1] == 112:
        dst[:, 0] += 8.0

    dst[:, 0] = dst[:, 0] * outsize[0] / target_size[0]
    dst[:, 1] = dst[:, 1] * outsize[1] / target_size[1]

    target_size = outsize

    margin_rate = scale - 1
    x_margin = target_size[0] * margin_rate / 2.
    y_margin = target_size[1] * margin_rate / 2.

    dst[:, 0] += x_margin
    dst[:, 1] += y_margin

    dst[:, 0] *= target_size[0] / (target_size[0] + 2 * x_margin)
    dst[:, 1] *= target_size[1] / (target_size[1] + 2 * y_margin)

    src = landmarks.astype(np.float32)

    tform = trans.SimilarityTransform()
    tform.estimate(src, dst)
    M = tform.params[0:2, :]
```

```
    aligned = cv2.warpAffine(img_rgb, M, (target_size[1], target_size[0]))

    if outsize is not None:
        aligned = cv2.resize(aligned, (outsize[1], outsize[0]))

    return aligned


def extract_aligned_face_fast(img_rgb: np.ndarray, res: int = 224, scale: float = 0.8) -> Optional[np.ndarray]:
    """
    얼굴 검출 및 정렬 (축소된 이미지에서 검출)
    - scale: 이미지 축소 비율 (0.8 = 80% 크기로 축소) -> time cost 감소
    - 얼굴이 없으면 None 반환
    """
    small = cv2.resize(img_rgb, None, fx=scale, fy=scale, interpolation=cv2.INTER_AREA)
    faces = face_detector(small, 1)

    if len(faces) == 0:
        return None

    face = max(faces, key=lambda r: r.width() * r.height())
    landmarks = get_5_keypoints(small, face)
    aligned = align_and_crop_face(small, landmarks, outsize=(res, res))

    return aligned
```

Utils - Frame Extraction

```
def uniform_frame_indices(total_frames: int, num_frames: int) -> np.ndarray:
    """비디오 프레임을 균등하게 샘플링"""
```

```
    if total_frames <= 0:
        return np.array([], dtype=int)
    if total_frames <= num_frames:
        return np.arange(total_frames, dtype=int)
    return np.linspace(0, total_frames - 1, num_frames, dtype=int)


def read_rgb_frames(file_path: Path, num_frames: int = NUM_FRAMES) -> List[np.ndarray]:
    """이미지 또는 비디오에서 RGB 프레임 추출"""
    ext = file_path.suffix.lower()

    if ext in IMAGE_EXTS:
        try:
            img = cv2.imread(str(file_path))
            if img is None:
                return []
            return [cv2.cvtColor(img, cv2.COLOR_BGR2RGB)]
        except Exception:
            return []

    if ext in VIDEO_EXTS:
        cap = cv2.VideoCapture(str(file_path))
        total = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))

        if total <= 0:
            cap.release()
            return []

        frame_indices = uniform_frame_indices(total, num_frames)
        frames = []

        for idx in frame_indices:
            cap.set(cv2.CAP_PROP_POS_FRAMES, int(idx))
            ret, frame = cap.read()
```

```
            if not ret:
                continue
            frames.append(cv2.cvtColor(frame, cv2.COLOR_BGR2RGB))

        cap.release()
        return frames

    return []
```

Data Preprocessing + Face Crop 저장

```
class PreprocessOutput:
    def __init__(
        self,
        filename: str,
        face_imgs: List[Image.Image],
        representative_face: Optional[np.ndarray] = None,
        error: Optional[str] = None
    ):
        self.filename = filename
        self.face_imgs = face_imgs  # PIL Images for inference
        self.representative_face = representative_face  # representative face save (RGB numpy)
        self.error = error


def preprocess_one_with_facecrop(file_path: Path, num_frames: int = NUM_FRAMES) -> PreprocessOutput:
    """
    파일 하나에 대한 전처리 수행 (얼굴 검출 + crop)
    - 비디오: 여러 프레임에서 얼굴 검출, 대표 1장 저장
    - 이미지: 1장에서 얼굴 검출
    """
```

```
    """
    try:
        frames = read_rgb_frames(file_path, num_frames=num_frames)

        if not frames:
            return PreprocessOutput(file_path.name, [], None, "No frames extracted")

        face_imgs: List[Image.Image] = []
        representative_face: Optional[np.ndarray] = None

        for i, rgb in enumerate(frames):
            aligned_face = extract_aligned_face_fast(rgb, res=224, scale=0.5)

            if aligned_face is not None:
                face_imgs.append(Image.fromarray(aligned_face))

                if representative_face is None:
                    representative_face = aligned_face

        if not face_imgs:
            return PreprocessOutput(file_path.name, [], None, "No face detected")

        return PreprocessOutput(file_path.name, face_imgs, representative_face, None)

    except Exception as e:
        return PreprocessOutput(file_path.name, [], None, str(e))
```

Step 1: Preprocessing & Saving Face Crop

```
files = sorted([p for p in TEST_DIR.iterdir() if p.is_file()])
print(f"Test data length: {len(files)}")
```

```
if SAVE_CROPS:
    print(f"Cropped faces will be saved to: {CROP_SAVE_DIR}")

preprocess_results: Dict[str, PreprocessOutput] = {}
no_face_files: List[str] = []
saved_count = 0

for file_path in tqdm(files, desc="Preprocessing"):
    out = preprocess_one_with_facecrop(file_path)
    preprocess_results[out.filename] = out

    if out.error and "No face" in out.error:
        no_face_files.append(out.filename)

    if SAVE_CROPS and out.representative_face is not None:
        save_name = Path(out.filename).stem + ".jpg"
        save_path = CROP_SAVE_DIR / save_name
        cv2.imwrite(
            str(save_path),
            cv2.cvtColor(out.representative_face, cv2.COLOR_RGB2BGR)
        )
        saved_count += 1

print("\nPreprocessing completed.")
```

```
Test data length: 500
```

```
Preprocessing: 100%|██████████| 500/500 [09:25<00:00,  1.13s/it]
```

```
Preprocessing completed.
```

```
# list of failed files - If you want to see files with no detected faces, uncomment below
'''
if no_face_files:
    print(f"\n=== Files with no face detected ({len(no_face_files)}) ===")
    for f in no_face_files[:30]:
        print(f"  - {f}")
    if len(no_face_files) > 30:
        print(f"  ... and {len(no_face_files) - 30} more")
'''
# results : missing faces data = 16
```

```
'\nif no_face_files:\n    print(f"\n=== Files with no face detected ({len(no_face_files)}) ===")\n    for f in no_face_files[:30]:\n
```

Model Load

```
print("Loading model...")
model = ViTForImageClassification.from_pretrained(MODEL_ID).to(DEVICE)
processor = ViTImageProcessor.from_pretrained(MODEL_ID)
model.eval()
```

```
Loading model...
```

```
ViTForImageClassification(
  (vit): ViTModel(
    (embeddings): ViTEmbeddings(
      (patch_embeddings): ViTPatchEmbeddings(
        (projection): Conv2d(3, 768, kernel_size=(16, 16), stride=(16, 16))
```

```
      )
      (dropout): Dropout(p=0.0, inplace=False)
    )
    (encoder): ViTEncoder(
      (layer): ModuleList(
        (0-11): 12 x ViTLayer(
          (attention): ViTAttention(
            (attention): ViTSelfAttention(
              (query): Linear(in_features=768, out_features=768, bias=True)
              (key): Linear(in_features=768, out_features=768, bias=True)
              (value): Linear(in_features=768, out_features=768, bias=True)
            )
            (output): ViTSelfOutput(
              (dense): Linear(in_features=768, out_features=768, bias=True)
              (dropout): Dropout(p=0.0, inplace=False)
            )
          )
          (intermediate): ViTIntermediate(
            (dense): Linear(in_features=768, out_features=3072, bias=True)
            (intermediate_act_fn): GELUActivation()
          )
          (output): ViTOutput(
            (dense): Linear(in_features=3072, out_features=768, bias=True)
            (dropout): Dropout(p=0.0, inplace=False)
          )
          (layernorm_before): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
          (layernorm_after): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
        )
      )
    )
    (layernorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
  )
  (classifier): Linear(in_features=768, out_features=2, bias=True)
)
```

```
def infer_fake_probs(pil_images: List[Image.Image]) -> List[float]:
    """PIL 이미지 리스트에 대해 Fake probability 추론"""
    if not pil_images:
```

```
    return []

probs: List[float] = []

with torch.inference_mode():
    inputs = processor(images=pil_images, return_tensors="pt")
    inputs = {k: v.to(DEVICE, non_blocking=True) for k, v in inputs.items()}
    logits = model(**inputs).logits
    batch_probs = F.softmax(logits, dim=1)[:, 1]  # Real probability (id2label: 0=Fake, 1=Real)
    probs.extend(batch_probs.cpu().tolist())

return probs
```

Step 2: Inference

```
results: Dict[str, float] = {}

for filename, out in tqdm(preprocess_results.items(), desc="Inference"):
    if out.face_imgs:
        probs = infer_fake_probs(out.face_imgs)
        results[filename] = float(np.mean(probs)) if probs else 0.0
    else:
        # 얼굴 검출 실패 시 0 (Real로 처리) -> basic logic
        results[filename] = 0.0
print("Done.\n")
```

```
Inference: 100%|██████████| 500/500 [00:05<00:00, 89.90it/s]


Done.
```

Submission

```
submission = pd.read_csv('./sample_submission.csv')
submission['prob'] = submission['filename'].map(results).fillna(0.0)

submission.to_csv(OUT_CSV, encoding='utf-8-sig', index=False)
print(f"Saved submission to: {OUT_CSV}")
```

```
Saved submission to: output/submission.csv
```


비전 torch insight DEEPFAKE DETECTIOIN

댓글 0개

SO softkleenex

comment

0 / 1000

작성

📣 댓글 작성 창의 위치가 댓글 리스트 상단으로 이동했습니다!

목록으로

이전 글	이전 글이 존재하지 않습니다.				
현재 글	[public 0.6] Baseline 코드 및 insight 공유 대회 - HAI(하이)! - Hecto AI Challenge : 2025 하반기 헥토 채용 AI 경진대회	좋아요 12	조회 914	댓글 0	7일 전
다음 글	간단하게 동영상을 frame로 처리하는 방법(실시간으로 colab에서 보는 법) 대회 - HAI(하이)! - Hecto AI Challenge : 2025 하반기 헥토 채용 AI 경진대회	좋아요 3	조회 471	댓글 0	7일 전

데이콘(주) | 대표 김국진 | 699-81-01021
통신판매업 신고번호: 제 2021-서울영등포-1704호
직업정보제공사업 신고번호: J1204020250004
서울특별시 영등포구 은행로 3 익스콘벤처타워 901호

이메일 dacon@dacon.io | 전화번호: 070-4102-0545